ENTREGA 3 · SOLUCIÓN DEL PROYECTO

Smart Finance

Gestión integral de finanzas personales con IA, local y privada



Mauro Alexander Romero Arias · Product Owner

Juan Camilo Alfonso Martínez · Scrum Master

Grupo 6 · Ingeniería de Software I – C2 · UIS · Prof. Duván Yahir Sanabria

El dinero está disperso y nadie lo controla

15 %

de los colombianos usa apps para administrar su dinero

SFC y CAF, 2019

12,5 / 21

puntaje del índice de educación financiera del país

Asobancaria, 2021

US$ 14,99

al mes cuesta YNAB; Mint cerró en 2024

Tabla de sistemas similares

Oportunidades que aprovechamos

Los bancos ya envían un SMS por cada compra
lo leemos y registramos el gasto

La IA entiende recibos y preguntas
escáner y asistente con tus datos reales

Desconfianza en la nube
todo se guarda en el teléfono

Objetivo y alcance

Objetivo general Desarrollar una app móvil multiplataforma (React Native + Expo, TypeScript, MVVM, SQLite local) con un servidor de IA protegido (Gemini) para que una persona gestione, planifique y entienda sus finanzas.

Incluido

- Cuentas en COP, USD y EUR
- Movimientos manuales, por voz, foto, CSV o SMS
- Presupuesto, metas, deudas, suscripciones y alertas
- Reportes PDF/CSV y asistente con IA
- Biometría, PIN, perfiles y respaldo
- Instaladores de prueba .ipa y .apk

Fuera del alcance

- Conexión directa con bancos (Open Finance requiere ser tercero autorizado)
- Sincronización en la nube entre dispositivos
- Publicación en App Store y Google Play
- Pagos o transferencias reales

13 módulos y 26 casos de uso implementados

RF1
Cuentas

RF2
Movimientos y transferencias

RF3
Escáner de recibos

RF4
Asistente IA

RF5
Suscripciones

RF6
Reportes PDF / CSV

RF7
Biometría y PIN

CU-008
Presupuesto con semáforo

CU-009
Metas de ahorro

CU-010
Deudas y préstamos

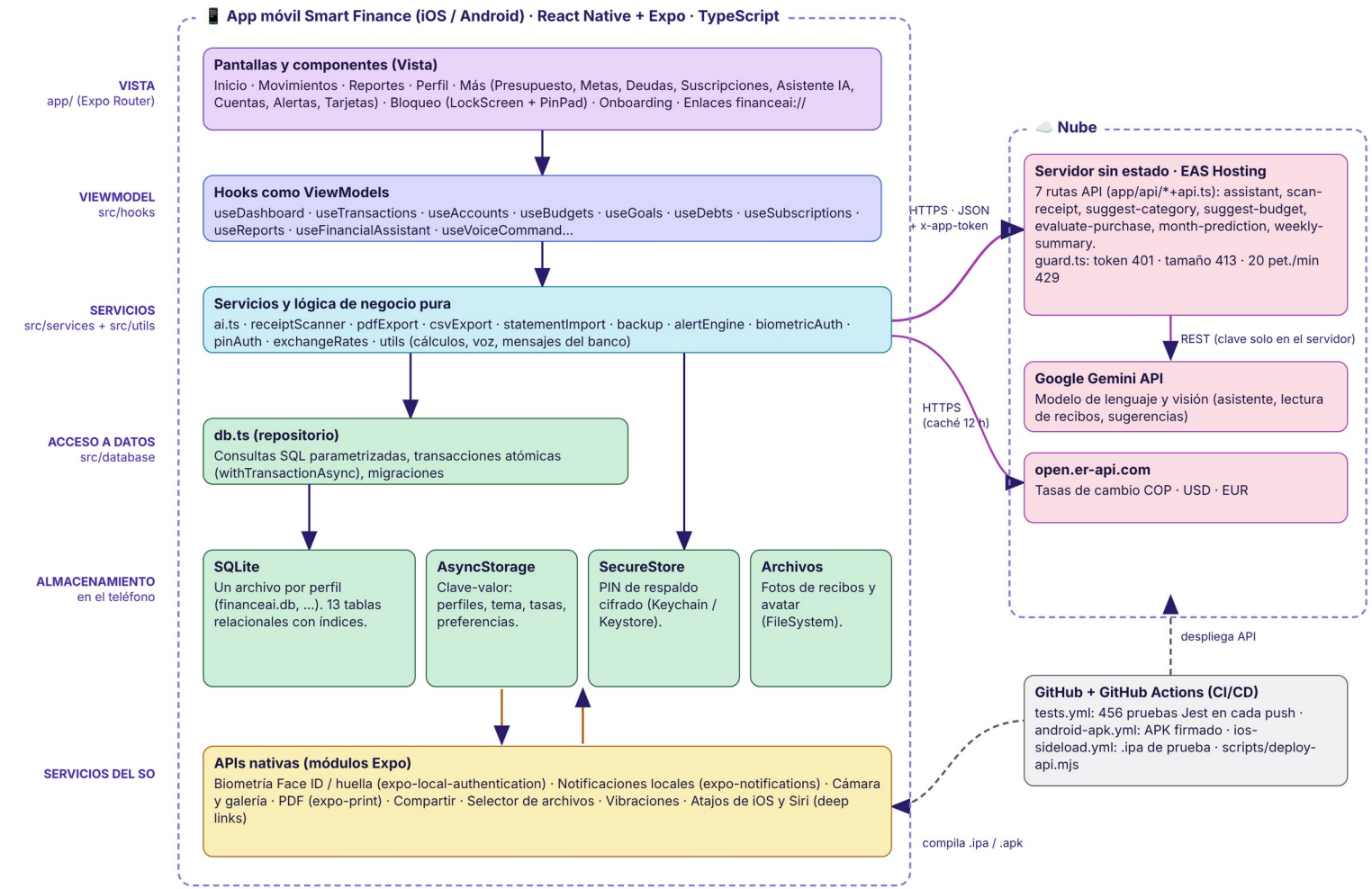
CU-011
Alertas

CU-018
Mensaje del banco

Sprint 6
Voz y Atajos de Siri

Morado: requerimientos funcionales RF1–RF7.
Durazno: funciones que ampliaron los RF en los Sprints 3 a 6.

Arquitectura local-first con MVVM



Flechas negras: dependencia entre capas (la vista solo conoce al ViewModel; solo db.ts conoce SQL). Flechas moradas: comunicación por red. Línea discontinua: integración y despliegue continuos.

Datos en el teléfono

Funciona sin internet, sin costo de nube, privado

Servidor sin estado

La clave de Gemini nunca está en la app; 401 · 413 · 429

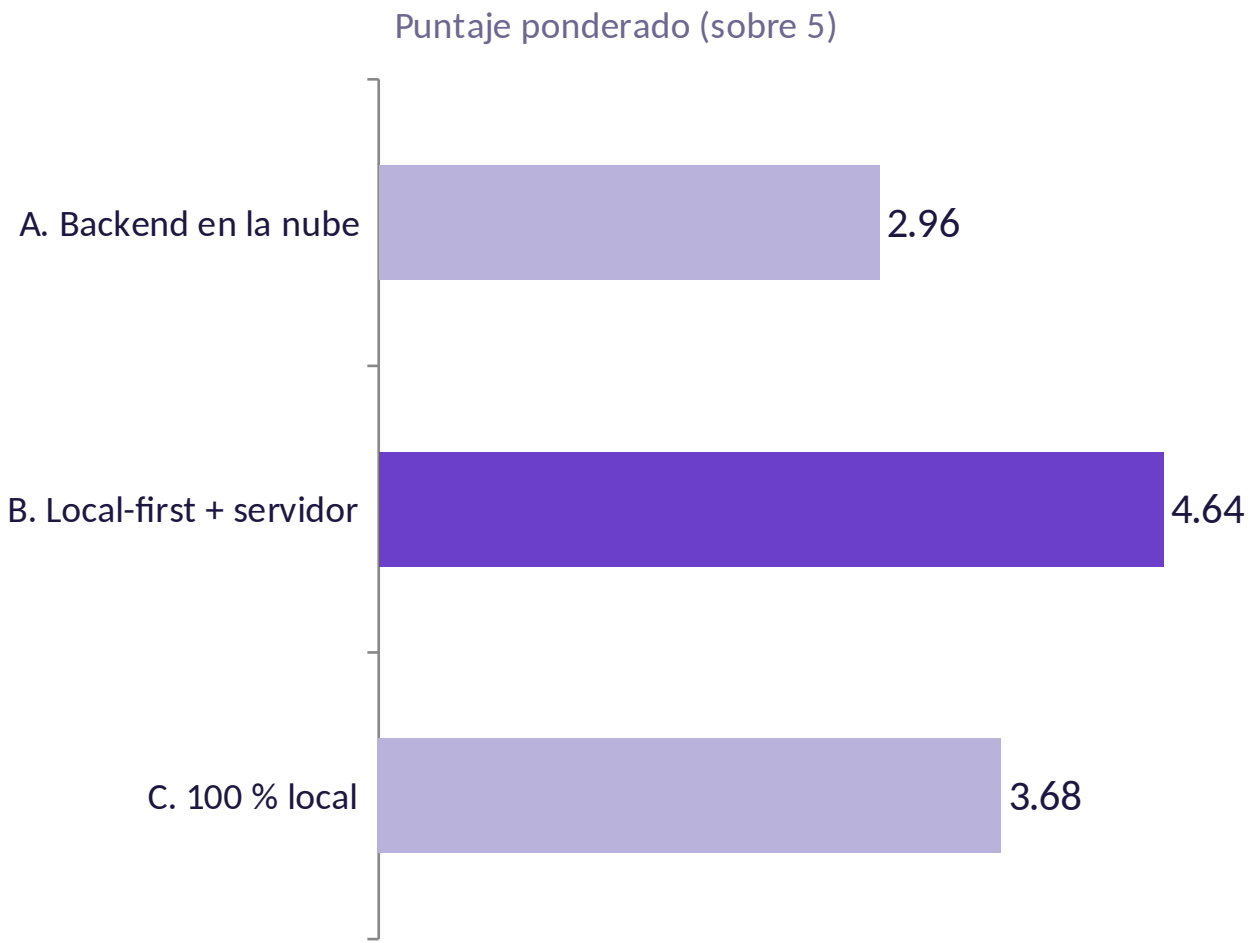
Capas MVVM

Lógica pura probada sin teléfono (92,8 %)

Transacciones atómicas

Un saldo nunca queda a medias

Tres soluciones evaluadas frente a los requisitos



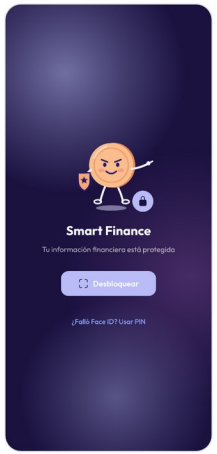
Criterios y peso

- Disponibilidad sin conexión (5)
- Seguridad de la clave de IA (5)
- Privacidad de los datos (5)
- Funciones de IA (4) · Costo (4)
- Esfuerzo para 2 personas (3)
- Sincronización entre equipos (2)

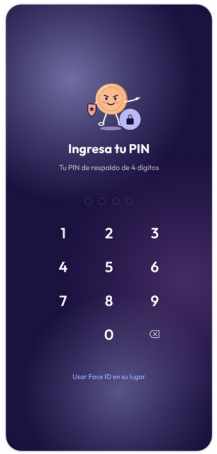
Compensación aceptada

B cede la sincronización automática a cambio de privacidad y funcionamiento sin internet. El respaldo JSON cubre el cambio de teléfono.

El producto en el iPhone



1. Bloqueo con Face ID / huella



2. PIN de respaldo



3. Bienvenida (onboarding)



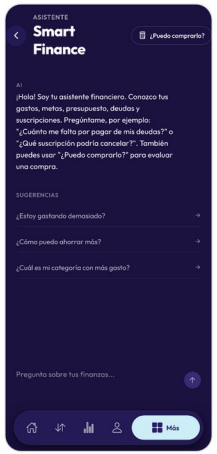
4. Inicio: saldo total y resumen



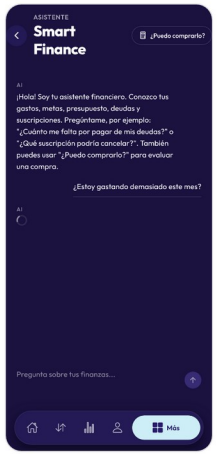
5. Movimientos con filtros



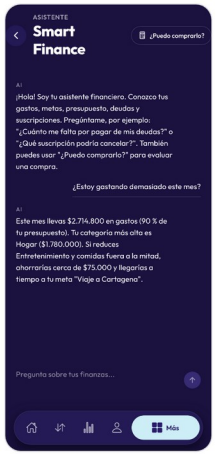
6. Perfil y configuración



1. Asistente financiero



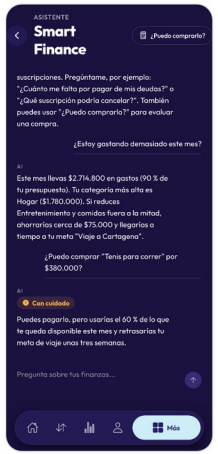
2. La IA analiza tus datos



3. Respuesta con datos reales



4. «¿Puedo comprarlo?»

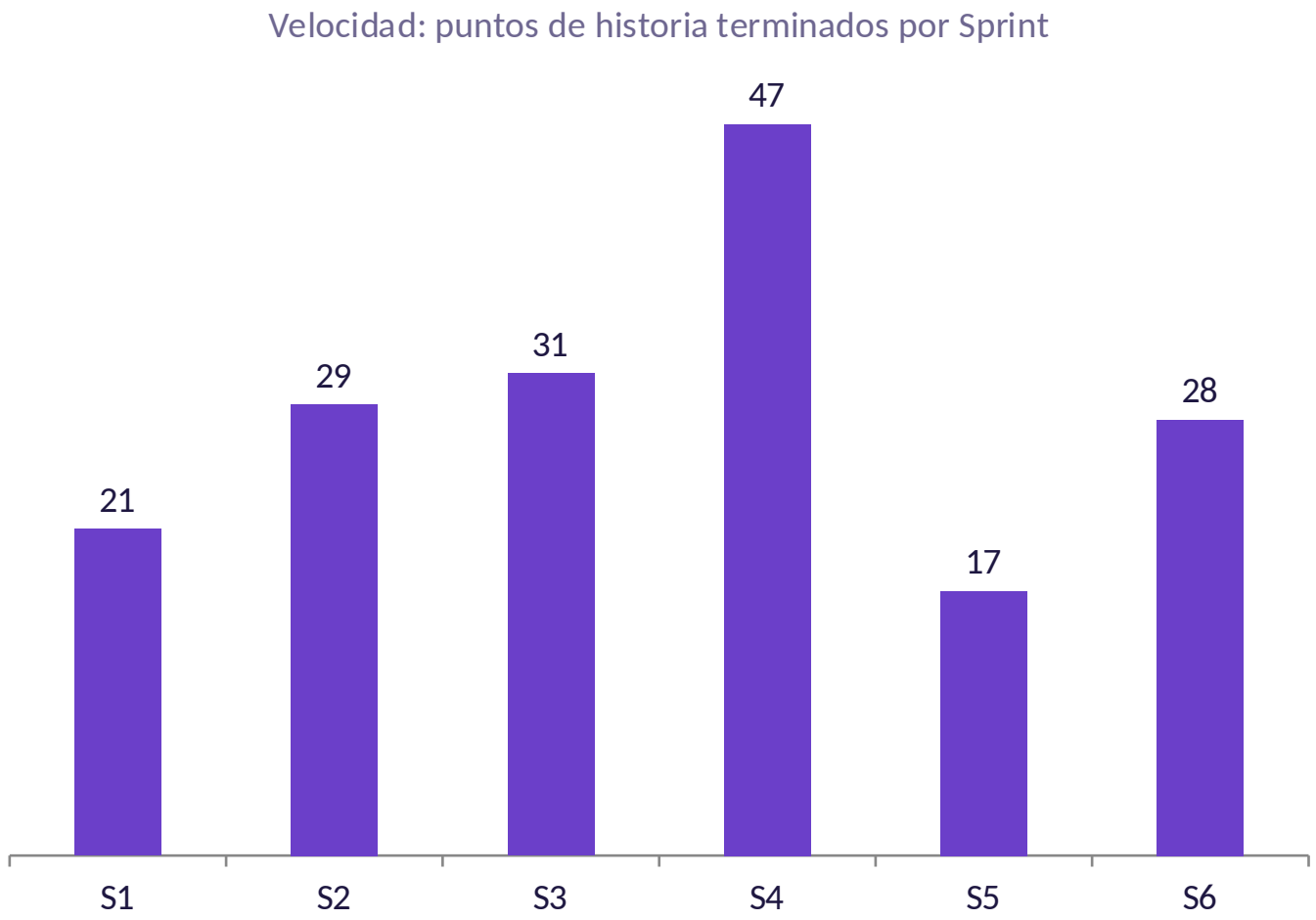


5. Veredicto de la compra



6. Inicio tras desbloquear

Seis Sprints con Scrum y Jira



173 / 201 puntos terminados (86 %)

320 h estimadas en la EDT por fases

56 commits en la rama principal

Decisiones que salieron de las Sprint Review

servidor para la IA (S4) · un archivo SQLite por perfil (S4) · lector de SMS y Atajos (S5) · prueba automática de rendimiento (S6)

Pruebas: funcionales, unitarias y de rendimiento

456 / 456

pruebas automáticas pasan (57 suites, Jest)

92,8 %

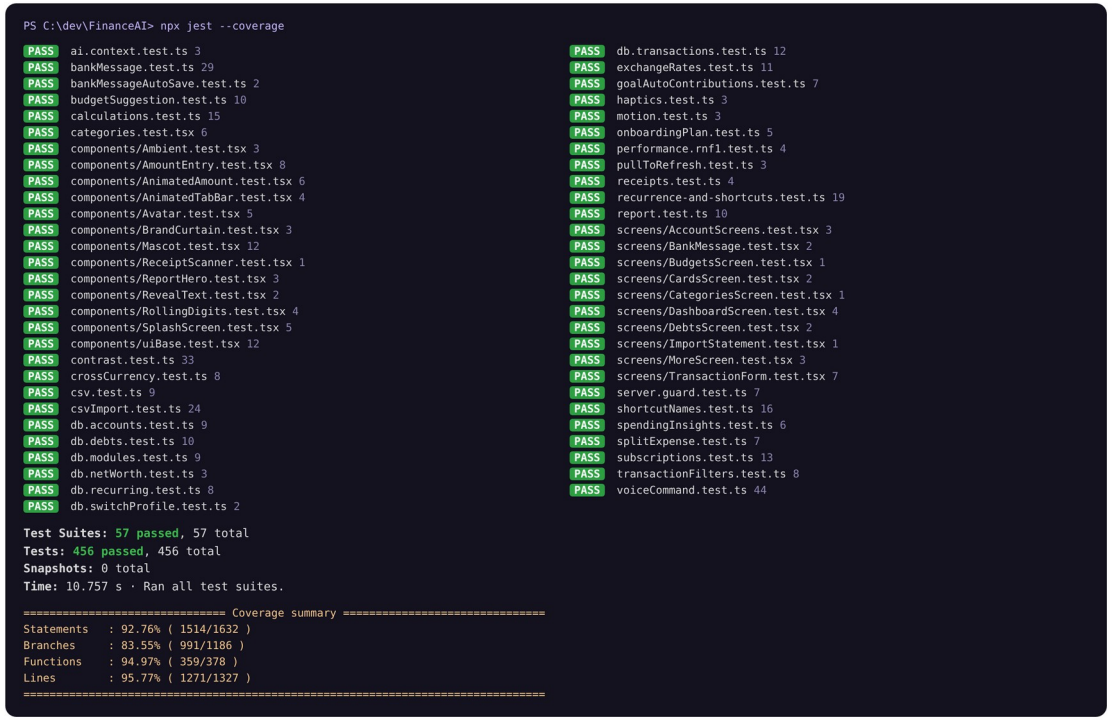
cobertura de sentencias de la lógica de negocio

25 / 25

pruebas funcionales cumplen

44 ms

cargar 5.000 movimientos (RNF1 pide < 2 s)



```
PS C:\dev\FinanceAI> npx jest --coverage

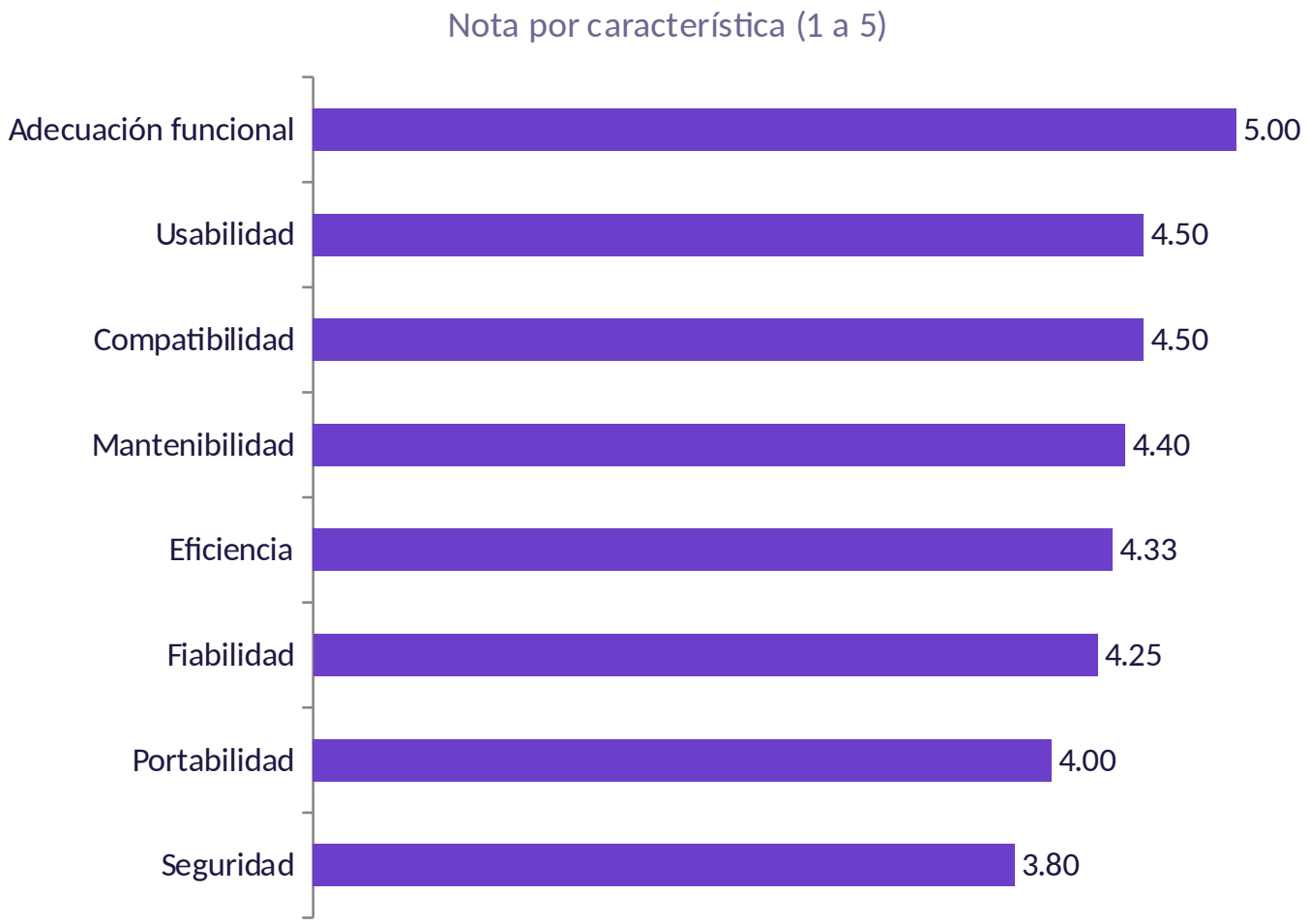
PASS ai.context.test.ts 3
PASS bankMessage.test.ts 29
PASS bankMessageAutoSave.test.ts 2
PASS budgetSuggestion.test.ts 10
PASS calculations.test.ts 15
PASS categories.test.tsx 6
PASS components/Ambient.test.tsx 3
PASS components/AmountEntry.test.tsx 8
PASS components/AnimatedAmount.test.tsx 6
PASS components/AnimatedTabBar.test.tsx 4
PASS components/Avatar.test.tsx 5
PASS components/BrandCurtain.test.tsx 3
PASS components/Mascot.test.tsx 12
PASS components/ReceiptScanner.test.tsx 1
PASS components/ReportHero.test.tsx 3
PASS components/RevealText.test.tsx 2
PASS components/RollingDigits.test.tsx 4
PASS components/SplashScreen.test.tsx 5
PASS components/uiBase.test.tsx 12
PASS contrast.test.ts 33
PASS crossCurrency.test.ts 8
PASS csv.test.ts 9
PASS csvImport.test.ts 24
PASS db.accounts.test.ts 9
PASS db.debts.test.ts 10
PASS db.modules.test.ts 9
PASS db.netWorth.test.ts 3
PASS db.recurring.test.ts 8
PASS db.switchProfile.test.ts 2
PASS db.transactions.test.ts 12
PASS exchangeRates.test.ts 11
PASS goalAutoContributions.test.ts 7
PASS haptics.test.ts 3
PASS motion.test.ts 3
PASS onboardingPlan.test.ts 5
PASS performance.rnf1.test.ts 4
PASS pullToRefresh.test.ts 3
PASS receipts.test.ts 4
PASS recurrence-and-shortcuts.test.ts 19
PASS report.test.ts 10
PASS screens/AccountScreens.test.tsx 3
PASS screens/BankMessage.test.tsx 2
PASS screens/BudgetsScreen.test.tsx 1
PASS screens/CardsScreen.test.tsx 2
PASS screens/CategoriesScreen.test.tsx 1
PASS screens/DashboardScreen.test.tsx 4
PASS screens/DebtsScreen.test.tsx 2
PASS screens/ImportStatement.test.tsx 1
PASS screens/MoreScreen.test.tsx 3
PASS screens/TransactionForm.test.tsx 7
PASS server.guard.test.ts 7
PASS shortcutNames.test.ts 16
PASS spendingInsights.test.ts 6
PASS splitExpense.test.ts 7
PASS subscriptions.test.ts 13
PASS transactionFilters.test.ts 8
PASS voiceCommand.test.ts 44

Test Suites: 57 passed, 57 total
Tests: 456 passed, 456 total
Snapshots: 0 total
Time: 10.757 s · Ran all test suites.

============================== Coverage summary ==============================
Statements   : 92.76% ( 1514/1632 )
Branches     : 83.55% ( 991/1186 )
Functions    : 94.97% ( 359/378 )
Lines        : 95.77% ( 1271/1327 )
==============================================================================
```

Módulos probados a fondo: db.ts (saldos y transacciones atómicas), presupuesto, lector de SMS, seguridad del servidor y comandos de voz.

Calidad ISO/IEC 25010: 4,36 sobre 5



13 / 13

requerimientos no funcionales cumplen su métrica

Notas más bajas y por qué

- No repudio (2): app de un solo usuario, sin registro de auditoría
- Instalación (3): sideload que se renueva cada 7 días
- Accesibilidad: falta auditoría con VoiceOver

Riesgos y compensaciones

Riesgo	P / I	Mitigación
Exponer la clave de la API de IA	A / A	Servidor intermedio con token
Perder datos al cambiar de teléfono	M / A	Respaldo y restauración JSON
Respuestas erróneas de la IA	M / M	Contexto real; el usuario confirma
Firma de Apple vence en la sustentación	A / M	Reinstalar el día antes y video
Saldos inconsistentes	B / A	Transacciones atómicas y pruebas

Compensaciones

Local-first
privacidad ⟷ sin sincronización

IA en la nube
calidad ⟷ requiere internet

MVVM con datos planos
pruebas fáciles ⟷ menos encapsulamiento

Integración por CSV/SMS
legal y gratis ⟷ menos automática

Conclusiones y trabajo futuro

OE1–OE6 cumplidos; OE7 parcial

El .ipa funciona en iPhone; el APK está en validación.

Analizar antes de programar sirvió

UML y la comparación de alternativas dieron decisiones que no cambiaron.

Las pruebas permitieron crecer

Agregamos funciones grandes al final sin romper las anteriores.

Scrum detectó problemas a tiempo

Firma de Apple, clave de IA y tildes en Atajos se resolvieron en un Sprint.

Trabajo futuro

- Sincronización cifrada iPhone ↔ Android
- Open Finance cuando la regulación lo permita
- Fotos de recibos en el respaldo
- Auditoría con VoiceOver y TalkBack
- Prueba de carga del servidor de IA
- Publicación en las tiendas

Gracias

¿Preguntas?



Repositorio: github.com/CamiloMaartinez/FinanceAI
Jira: cm942995.atlassian.net · proyecto SF
Video de funcionalidades: incluido en el entregable Grupo6